

No Evidence That TaperNorm Increases Forgetting After a Domain Shift: Two Pre-Specified Studies

Daniel Shdeed
Independent researcher
danielshdeed@gmail.com

October 2026

Abstract

TaperNorm replaces a pre-norm Transformer’s internal normalization with a gated map that acts like RMSNorm early in training and then becomes a fixed linear scaling. Without per-token normalization, a model might forget more after a shift in the training data. Two small studies tested this, with designs, endpoints and decision rules fixed before training. Paired 17.7M-parameter models with internal RMSNorm or TaperNorm were trained on 150M web tokens, then continued for 100M tokens on web text or a new domain. The endpoint D is a difference-in-differences in held-out web cross-entropy; positive D means extra forgetting under TaperNorm. In a pilot with Python code (three seeds), mean D was -0.022 nats/token, below the pre-specified $+0.015$ bound, so the decision was to stop. A second study, whose frozen protocol and code were hashed and submitted for timestamping before it ran, chose a new domain by a fixed rule (Chinese web text) and added three fresh seeds. On Chinese, mean D over six seeds was -0.033 (95% interval -0.068 to $+0.003$), just past the -0.03 threshold for the opposite direction; a post hoc one-sided 95% upper bound of -0.005 rules out an excess of $+0.015$. Under GPT-2’s tokenizer, however, 85% of Chinese labels are byte fragments, so much of that shift concerns which output tokens must be predicted, which the internal normalizers do not touch. On the fresh seeds the Python decision replicated (mean D $+0.009$), but D was positive in all three, where it had been negative in all three pilot seeds: three seeds understated seed variation. A pre-specified in-training measure of the scale shift behind the hypothesis was about four times larger for Chinese than for Python, yet TaperNorm did not forget more. Neither study showed equivalence within ± 0.015 . We find no evidence that TaperNorm increases forgetting under these shifts. Code and all records are released.

1 Introduction

Normalization layers such as LayerNorm (Ba et al., 2016) and RMSNorm (Zhang and Sennrich, 2019) are a standard part of Transformer language models. In the common pre-norm arrangement (Xiong et al., 2020), every attention and MLP sub-layer reads its input through a normalizer that rescales each token’s activation vector by that token’s own statistics. A long line of work has asked whether normalization is necessary at all (Zhang et al., 2019b; Brock et al., 2021; He and Hofmann, 2024; Zhu et al., 2025). TaperNorm (Kanavalau et al., 2026) is a recent proposal for Transformers; we follow version 1 of its arXiv preprint throughout. It acts exactly as RMSNorm (or LayerNorm) during an initial gate warm-up, calibrates a fixed scaling branch during that warm-up, and then cosine-decays a global gate until each internal normalizer becomes a sample-independent linear map. That map can be folded into the adjacent linear projections at inference. Kanavalau et al.

(2026) report that this matches normalized baselines under identical setups, and they measure up to $1.22\times$ higher throughput in an inference microbenchmark.

This study looks at a property that the original evaluation did not target: behaviour under a later shift in the training distribution. A normalizer is invariant to the scale of its input. If continued training on a new domain changes activation magnitudes, an RMSNorm layer absorbs part of that change automatically, whereas a tapered layer applies a scale calibrated on the earlier distribution. One can conjecture that the tapered network must then absorb the shift through weight updates, which might overwrite structure that supports the original domain and so increase catastrophic forgetting (McCloskey and Cohen, 1989; French, 1999; Kirkpatrick et al., 2017). This argument motivates the hypothesis; the experiment tests the hypothesis, not the mechanism.

Hypothesis. H1: after training switches from web text to Python code, internal TaperNorm (without the auxiliary scale loss; final RMSNorm retained) increases the persistent deterioration of held-out web cross-entropy, relative to continuing web training, compared with internal RMSNorm. *Persistent* is defined in Section 3.1.

We report two studies. **Study 1** is a deliberately small, inexpensive pilot that asked whether H1 deserves a larger study (Sections 3 to 5). Its design pairs the two conditions on initialization and data order and branches each condition’s trained prefix into a web continuation and a Python continuation from an identical saved state. It fixes a difference-in-differences endpoint and an ordered set of decision rules before any training. Its pre-specified decision was to stop. A pilot with one domain pair and three seeds says little about the hypothesis in general, and an exploratory analysis of the pilot suggested that the premise of H1, a domain-induced change in activation scale, was barely present for Python. We therefore ran **Study 2** (Section 6), a separate study whose protocol and code were frozen, and their hash submitted for timestamping, before any of its jobs ran. It chose a new domain by a fixed rule and tested H1’s direction there on six seeds (**H2**). It also replicated Study 1’s web-to-Python contrast on three fresh seeds (**R2**) and measured the scale premise directly as a pre-specified manipulation check. Our contributions are:

- a paired, pre-specified design for measuring how an architectural choice affects forgetting under a domain shift, whose primary contrast decomposes exactly into branch-level forgetting, a control-branch term, per-document effects and per-token-class effects;
- Study 1’s result. All validity checks pass and the pre-specified decision is *stop: small observed effect*. Mean $D = -0.0223$ nats/token, against a $+0.015$ bound for a meaningful excess;
- Study 2’s results. The selected domain is Chinese web text. There, mean D_X over six seeds was -0.0325 (95% interval $[-0.0683, +0.0032]$), which falls just past the pre-specified -0.03 threshold into the *opposite direction* category; that label depends on single seeds, but the pre-specified reading does not. On the fresh seeds the web-to-Python result replicated (*stop*; mean $D = +0.0093$). A pre-specified in-training measure of the premise, the domain-specific change in activation scale at the normalizer inputs, was 4.3–4.4 times larger for Chinese than for Python, yet TaperNorm did not forget more;
- a cautionary measurement of seed variation. Study 1’s three paired seeds agreed closely (SD 0.0034), and three fresh seeds then gave web-to-Python contrasts of the opposite sign; the six seeds’ SD is about 5 times Study 1’s;
- a lightweight procedure for fixing a cloud-executed study’s plan before it runs without a prior registry entry, with an account of what it does and does not establish. The frozen protocol and code are hashed and the hash timestamped, every cloud notebook embeds the hash, and

saved model states that are reused across sessions are re-evaluated bit-exactly against their own records before they train;

- code, frozen configurations, data hashes and all evaluation and training records, sufficient to recompute every number in this paper.

Both studies are small (17,718,784 parameters, at most six seeds, two domain pairs) and should be read as screening results, not as a general statement about TaperNorm (Section 8).

2 Related work

Normalization and its removal. LayerNorm (Ba et al., 2016) and RMSNorm (Zhang and Sennrich, 2019) normalize each token independently, and the pre-norm placement (Xiong et al., 2020) is now standard in decoder-only language models. Fixup (Zhang et al., 2019b) and NFNets (Brock et al., 2021) train deep residual networks without normalization through initialization, scaling and clipping choices. For Transformers, He and Hofmann (2024) study which standard components of the block can be removed, and Zhu et al. (2025) replace normalization layers with an element-wise tanh map. TaperNorm (Kanavalau et al., 2026) instead removes internal normalization gradually during training. Its analysis attributes the essential role of normalization to the *output* normalizer, which anchors scale; this is why our Taper-minus condition keeps the final RMSNorm.

Normalization, weight decay and effective learning rate. When a normalizer makes a network’s output invariant to the scale of some weights, weight decay no longer regularizes those weights in the usual sense; it shrinks them and thereby raises their effective learning rate (van Laarhoven, 2017; Zhang et al., 2019a). Dohare et al. (2024) document a loss of plasticity in deep continual learning, and Lyle et al. (2024) connect normalization and effective learning rates to the loss of plasticity under nonstationary training. These results bear on our exploratory scale analysis (Section 5.4).

Forgetting under continued training. Catastrophic forgetting (McCloskey and Cohen, 1989; French, 1999) is the loss of earlier competence when a network is trained on new data, and it has motivated methods such as elastic weight consolidation (Kirkpatrick et al., 2017). The training regime, including learning-rate decay, affects how much a network forgets (Mirzadeh et al., 2020); larger pre-trained models forget less (Ramasesh et al., 2022); and the choice of normalization layer matters in online continual learning (Pham et al., 2022). In language modelling, continued pre-training on a new domain improves in-domain performance (Gururangan et al., 2020), but it can degrade performance on the original distribution; Gupta et al. (2023) and Ibrahim et al. (2024) study how learning-rate re-warming, re-decaying and replay trade adaptation against this forgetting. Our design deliberately uses no replay, re-warming or other mitigation. It isolates how much forgetting the normalization choice itself causes, relative to a matched control branch.

Pre-specification. Fixing hypotheses, endpoints and analysis rules before seeing outcomes protects against flexible analysis (Nosek et al., 2018). Neither study was lodged with a registry before it ran. Sections 3.6 and 6.3 state exactly what was fixed, and when, and what evidence supports each claim. For Study 2 that evidence includes a hash of the frozen protocol and code, timestamped through OpenTimestamps (Todd, 2016), which anchors digests in the Bitcoin blockchain without revealing the document.

3 Study 1: design

3.1 Notation and estimands

We write u for the global optimizer update ($u = 1, \dots, 15,260$) and $s = u - 9,156$ for the continuation update within a branch. Two different events fall on the number 6,104: the TaperNorm gate reaches zero at global update $u = 6,104$, and the primary endpoint is continuation update $s = 6,104$ (global update $u = 15,260$). They coincide because the protocol ends the taper at 40% of the 15,260-update trajectory, which equals the continuation length. We always state whether a number is a u or an s .

Let $L(m, b, s)$ be the token-weighted cross-entropy on the held-out web development set (nats per supervised token, 2,097,152 labels), for normalization condition $m \in \{\text{R}, \text{T}\}$ (RMS, Taper-minus), after s continuation updates on branch $b \in \{\text{web}, \text{code}\}$. Let $L(m, \text{prefix})$ be the same quantity at the switch. The primary endpoint is

$$D(s) = [L(\text{T}, \text{code}, s) - L(\text{T}, \text{web}, s)] - [L(\text{R}, \text{code}, s) - L(\text{R}, \text{web}, s)], \quad (1)$$

evaluated at $s = 6,104$. Positive D means that, relative to its own web-continuation control, the Taper-minus model loses more web performance under the Python shift than the RMS model does. Within each condition the code-minus-web difference cancels everything the two branches share, including the condition-specific quality of the prefix and generic continued-training drift.

Persistent. We call a deterioration *persistent* if it is present at the fixed endpoint $s = 6,104$, after 100M continuation tokens and at the end of the learning-rate schedule. A difference visible only early in continuation ($s \leq 1,000$) is *transient*, which the decision rules treat separately (Table 1, rule 5).

We also report the actual forgetting $F(m, b) = L(m, b, 6104) - L(m, \text{prefix})$ and the change in the between-condition gap during web continuation, $G_{\text{web}} = [L(\text{T}, \text{prefix}) - L(\text{R}, \text{prefix})] - [L(\text{T}, \text{web}, 6104) - L(\text{R}, \text{web}, 6104)]$. These satisfy the identity

$$D = \underbrace{F(\text{T}, \text{code}) - F(\text{R}, \text{code})}_Q + G_{\text{web}}, \quad (2)$$

so $Q = D - G_{\text{web}}$ is the direct difference in Python-branch forgetting. The identity is checked numerically in the analysis code. Because CE is token-weighted, D decomposes exactly into per-document terms D_i (weighted by each document’s scored-label count) and into four disjoint GPT-2 token classes: whitespace (W), text containing an alphanumeric character (A), other punctuation and symbols (P), and special, control or invalid-token tokens (C). A fifth, overlapping flag R marks rare tokens: those that occur fewer than 100 times in that seed’s prefix training stream.

3.2 Models and conditions

All models are pre-norm decoder-only Transformers adapted from nanoGPT (Karpathy, 2022):

- 6 layers, width 256, 4 heads of size 64, and an exact-GELU MLP of width 1,024;
- learned absolute positions, context 512, no biases and no dropout;
- GPT-2 token embeddings (Radford et al., 2019), tied to the output layer (vocabulary 50,257).

The RMS model has 17,718,784 parameters; Taper-minus adds 3,072 for its second gain vectors. About 73% of all parameters are in the token-embedding table. Both conditions keep the final RMSNorm.

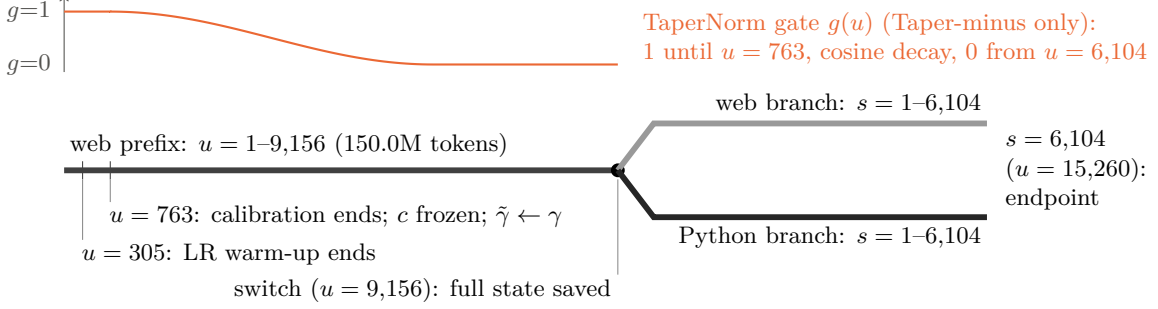


Figure 1: One condition and seed of the design (u : global update; s : continuation update). Both conditions start from identical initial weights and read the same data order for each seed. At the switch ($u = 9,156$), each condition’s complete state (weights, optimizer moments, loss scale and random-number state) is saved, and both of its branches restore that identical state. The web branch continues on C4; the Python branch trains on Python files from The Stack. The global learning-rate clock continues through the branches without a restart. Evaluations follow the schedule in Appendix A.

Initialization. All linear and embedding weights (token and position) are drawn from $\mathcal{N}(0, 0.02^2)$. The two residual output projections of each block (the attention output projection and the MLP down-projection) use $\mathcal{N}(0, (0.02/\sqrt{12})^2)$, where $12 = 2 \times 6$ layers. All gains start at 1.

RMS. All 12 internal normalizers are RMSNorm with $\epsilon = 10^{-6}$.

Taper-minus. All 12 internal normalizers are TaperNorm, following the operator and calibration of Kanavalau et al. (2026) (their Section 3 and Appendix C), without their auxiliary scale loss. For an input $h \in \mathbb{R}^d$ at global update u , with gains γ and $\tilde{\gamma}$,

$$r(h) = \sqrt{\|h\|_2^2/d + 10^{-6}}, \quad y = g(u) \frac{h}{r(h)} \odot \gamma + (1 - g(u)) c h \odot \tilde{\gamma}. \quad (3)$$

The scalar c of each site is calibrated during updates $u = 1$ – 763 as the ratio of bias-corrected exponential moving averages (one EMA step per optimizer update, new-sample weight 0.01) of $\|h \odot \gamma\|^2/r(h)$ and $\|h \odot \tilde{\gamma}\|^2$. These are collected from the pre-step forward passes of every microbatch. After update $u = 763$, c is frozen and $\tilde{\gamma}$ is initialized from γ ; $\tilde{\gamma}$ first receives an optimizer step at $u = 764$ and stays trainable. The gate is $g(u) = 1$ for $u \leq 763$, $g(u) = \frac{1}{2}[1 + \cos(\pi(u - 763)/5341)]$ for $763 < u < 6104$, and $g(u) = 0$ for $u \geq 6104$. At $g = 0$ no per-token statistic is computed and γ receives no gradient. Until $u = 763$ the two conditions are mathematically identical. The gate timing, GELU, learned positions, weight decay, learning-rate schedule and domain branching are choices of this study, so this is an adaptation of TaperNorm, not a replication of its experiments.

3.3 Training protocol

Optimization. The optimizer is AdamW (Loshchilov and Hutter, 2019) with $\beta = (0.9, 0.95)$ and $\epsilon = 10^{-8}$. All two-dimensional parameters, embeddings included, get weight decay 0.1; gain vectors get none. Gradients are clipped once per update to global norm 1.0, after accumulation. Each update uses $32 \times 512 = 16,384$ supervised tokens. The learning rate rises linearly to 6×10^{-4} over the first 305 updates and then follows a cosine decay to 6×10^{-5} at $u = 15,260$. The branches continue this global clock, with no learning-rate restart, optimizer reset, replay or regularization toward the prefix (Figure 1).

Pairing. Seeds 101, 102 and 103 are paired across conditions. A canonical RMS initialization is generated from the seed and explicitly copied into the Taper-minus model; $\tilde{\gamma}$ starts at 1 and is inactive until $u = 764$. Both conditions read the same pre-generated window order. Each run consists of a prefix of 9,156 updates (150,011,904 tokens) followed by two branches of 6,104 updates each (100,007,936 tokens). The web branch reads further unseen web windows and the Python branch reads Python windows; no window repeats within a trajectory. The primary allocation is 2.10B supervised tokens.

Precision. Training ran in FP16 autocast with dynamic loss scaling (Mickevicus et al., 2018). Master weights, optimizer moments, normalization and calibration statistics, and the loss were kept in FP32. A step whose gradients were nonfinite was to be retried on the same batch at a halved loss scale, so no update is ever skipped. This replaced the protocol’s original BF16 plan, because the available hardware (NVIDIA T4) lacks efficient BF16 support (Table 2).

3.4 Data

Sources and splits. Web text comes from C4 English (Raffel et al., 2020). Training uses the official training split, and the official validation split is hashed 50/50 into development and reserved-test sets. Code comes from the Python subset of The Stack (deduplicated) (Kocetkov et al., 2022). The dataset revisions are 1588ec454efa (C4) and 17cad72c886a (The Stack). Python files are grouped into connected components over their repository-name aliases (140,195 groups). Each group is assigned to a split by the value of $\text{sha256}(\text{group}) \bmod 100$: 0–89 train, 90–94 development and 95–99 test. Empty documents, and Python files without a usable repository name, were dropped.

Deduplication. Documents are deduplicated exactly by content hash across all domains and splits. An approximate near-duplicate search uses MinHash (Broder, 1997) over GPT-2 token 5-grams (128 permutations, LSH with 32 bands of 4 rows), confirmed by exact Jaccard similarity ≥ 0.85 . When duplicates straddle splits, the copy in the higher-priority split is kept (test over development over training). This removed 0 exact and 2,182 near duplicates. Documents shorter than five tokens have no 5-gram and are covered only by exact deduplication (20 documents).

Recall of the near-duplicate search. With 32 bands of 4 rows, a pair with Jaccard similarity J becomes an LSH candidate with probability $1 - (1 - J^4)^{32}$. A pair at $J = 0.85$ is therefore missed with probability 5.5×10^{-11} , and a pair at $J = 0.70$ is still found with probability 0.9998. This analytic recall is the evidence that near duplicates above the threshold were found. The preparation code also compared a deterministic random sample of 2,043 cross-split document pairs by exact Jaccard, and none reached 0.85. Because two randomly chosen documents are almost never near duplicates, that comparison is a sanity check with negligible power to detect leakage, not evidence of its absence.

Packing and quotas. Text is tokenized with GPT-2 BPE, with one EOS appended per document, and packed into 513-token windows at stride 512 with no padding. Windows are packed across document boundaries without an attention mask: a window can hold the end of one document, an EOS token and the start of the next, and attention can cross the EOS. Positions restart at 0 in every window. Each label is attributed to its own document (a label after an EOS belongs to the new document), so per-document sums reconstruct the corpus CE exactly. Quotas are

Table 1: Pre-specified decision rules, applied in order. CE values are in nats/token; a bar denotes the mean over seeds; s is the continuation update.

#	Category	Condition
1	Invalid or incomplete	Missing seed group, unresolved correctness or data failure, nonfinite primary run, or resource termination
2	Comparability / adaptation limited	Any seed’s relative prefix web-CE gap $> 2\%$, or either condition’s Python branch improves non-W code CE by < 0.05
3	Opposite direction	$\bar{D}(6104) \leq -0.03$
4	Proceed to design next study	$\bar{D}(6104) \geq 0.03$; $D(6104) > 0$ in every seed; $\bar{D}(3050) \geq 0.015$; $\bar{Q} > 0.5 \bar{D}$; mean matched differential forgetting > 0 at the furthest common target
5	Transient only	$\bar{D}(6104) < 0.015$ and quick-dev $\bar{D}(s) \geq 0.05$ at some scheduled $s \leq 1000$
6	Stop: small observed effect	$\bar{D}(6104) < 0.015$
7	Inconclusive	Anything else

260,000,257 web and 110,000,129 Python training tokens, and 2,097,152 development labels (4,096 windows) per domain. The *quick* development set is the first 262,144 of those labels. Reserved-test sets of 8,388,608 labels per domain were sealed and never used. The non-whitespace code CE used to measure adaptation excludes class W only.

3.5 Evaluation schedule and decision rules

Full development evaluations ran at 13 prefix points and at 25 continuation points per branch. Quick development evaluations ran at the switch and at 80 continuation points per branch, dense early on (Appendix A). Every full evaluation stores per-document and per-class sufficient statistics.

The secondary *matched-adaptation* analysis is fixed in advance. It fixes targets at the RMS Python branch’s quick-dev code CE at $s \in \{1525, 3050, 6104\}$. It finds the first downward crossing of each target by Taper-minus within an observed bracket of at most 100 updates (no extrapolation) and interpolates Taper-minus’s web CE at that crossing. It then compares the two models’ web forgetting relative to their own $s = 0$.

The decision rules in Table 1 are applied in order once all three seed groups are complete. They are screening criteria, not significance tests. With three seeds, the reported 95% interval $\bar{D} \pm 4.303 s_D / \sqrt{3}$ is descriptive.

Choice of thresholds. The protocol fixed the thresholds of 0.015 and 0.03 nats/token in advance as investment thresholds for a screening pilot. It records no power analysis or formal derivation for them, and we do not supply one after the fact. For orientation, a cross-entropy difference of 0.015 or 0.03 nats/token corresponds to a 1.5% or 3.0% change in perplexity ($e^{0.015} \approx 1.015$, $e^{0.03} \approx 1.030$). A mean between the two thresholds falls into the inconclusive category rather than into either decision.

3.6 Pre-specification, timeline and amendments

Protocol. The hypothesis, design, estimands, evaluation schedule and decision thresholds come from a written protocol (“protocol v3”), drafted on 12 September 2026, before the corpus was prepared or any model was trained on it. The protocol text is included in the project repository.

It was not lodged with an external registry, which is a limitation (Section 8). The amendments in Table 2 were made before any primary-run update; none of them changes the model, data, token budgets, seeds, endpoint or thresholds.

Timeline of the primary run (3 October 2026, UTC).

- 12:18: the run starts.
- 12:19:23: the run log records the frozen configuration hash (`c18441799eee2b6a...`). The hashed configuration contains the decision thresholds, and the hash can be recomputed from the released `config.json` by the runner’s own procedure.¹
- 14:25–15:26: seed 101’s four endpoint cross-entropies (both conditions, both branches) appear in the run’s live log. Seed 101’s D could have been computed from them.
- 16:07: commit `3ee1dc4` is made, containing the protocol text and the runner script.
- 21:40: the run ends, and the same process applies the decision rules and writes the decision report.

Because the commit came after seed 101’s endpoint values had been printed, the commit on its own does not establish pre-specification, and we disclose that here and in Section 8. The evidence that the analysis was fixed in advance is different. The thresholds used by the run were bound into the configuration hash logged at 12:19:23, before any outcome existed. The runner script, whose decision thresholds appear on line 82, has the same SHA-256 (`66769e7de30ec36f...`) in every commit of the repository history, and that hash matches the one recorded by the run itself. The decision report was produced by that same process.

Engineering runs. Before the primary run, engineering runs measured throughput and verified checkpoint resume. All of them were discarded:

- local development runs on stand-in corpora (WikiText-103 and CodeSearchNet functions), which trained RMS models only;
- preflights of an earlier execution pipeline under engineering seed 997, which timed each training phase by repositioning the update clock rather than by training a prefix to the switch;
- a smoke test of about 20 minutes on the final runner, which repeated the first 2,197 (Taper-minus) and 2,336 (RMS) prefix updates of seed 101.

None of them produced a Taper-minus-versus-RMS continuation contrast on the C4 and Stack data, so no value of D was computed before the primary run.

4 Study 1: execution and implementation checks

Study 1’s primary run took one 9.36-hour session on a cloud machine with two NVIDIA Tesla T4 GPUs (PyTorch 2.11.0, CUDA 12.8), with RMS on one GPU and Taper-minus on the other. All six condition–seed runs completed all 21,364 updates. There were no nonfinite losses, gradients or

¹The hash is *not* the SHA-256 of the file itself; `sha256sum config.json` gives a different value. To reproduce it, load the JSON, remove the keys `config_sha256` and `checkpoint_seconds`, serialize the rest as canonical JSON (keys sorted, separators `","` and `":"`, no whitespace) and take the SHA-256 of the UTF-8 bytes. `paper/README.md` gives a one-line command, and `paper/make_assets.py` asserts the match on every build.

Table 2: Protocol amendments, all made before the primary run.

Date	Amendment
2026-09-28	NVIDIA T4 hardware: FP16 autocast with dynamic loss scaling replaces BF16; FP32 master weights, moments, normalization and EMA statistics, and loss. Results are not pooled with any BF16 run.
2026-09-30	A cap of 50 free cloud GPU-hours, spread over sessions and weekly quota resets, replaces the original 24-GPU-hour paid budget; the scientific allocation is unchanged.
2026-10-01	RMS and Taper-minus are trained as independent processes on two GPUs, with no shared state.
2026-10-03	Microbatch of 8 sequences $\times$ 4 accumulation steps (the protocol’s default layout; effective batch unchanged). For storage, model weights are kept only for the switch state and branch endpoints; the per-document and per-class statistics required at every full-development point are kept.

Table 3: Share of updates with gradient clipping active (pre-clipping global norm > 1.0), mean over seeds with the range across seeds in parentheses, by training phase. u is the global update.

Phase	Updates	RMS	Taper-minus
Prefix, $u \leq 763$ (calibration)	763	15.6% (15.3–16.0)	15.6% (15.3–16.0)
Prefix, $764 \leq u \leq 6,103$ (gate decaying)	5,340	4.3% (1.8–6.4)	10.9% (6.5–17.2)
Prefix, $6,104 \leq u \leq 9,156$ (gate zero)	3,053	15.4% (8.8–22.1)	49.9% (38.1–66.6)
Web branch	6,104	88.4% (83.1–93.1)	95.3% (92.5–98.4)
Python branch	6,104	95.1% (92.7–97.6)	95.9% (94.8–97.3)

weights; the loss scale grew to 16,384 and never needed a retry (0 retries in total). GPU kernels used for the backward pass are nondeterministic, so results are reproducible to within numerical noise, not bitwise.

Gradient clipping. Clipping at global norm 1.0 is part of the same optimization recipe for both conditions; we report how often it was active, per phase, from the per-update training logs (Table 3). During calibration, where the two conditions are mathematically identical, clipping was active in 16% of updates for both, as it should be. While the gate decayed it was less frequent (4% of updates for RMS, 11% for Taper-minus, as means over seeds). After the gate reached zero it became more frequent, more so for Taper-minus (50% vs 15% of the remaining prefix updates). On both continuation branches it was active in 88–96% of updates in both conditions. We report these rates descriptively and did not analyse their consequences.

Implementation checks. The training code is a single script that transcribes a separately tested reference implementation of the model, optimizer step and analysis. Before the run, it was checked against that reference in seven ways:

- (i) identical learning-rate and gate values at all 15,260 updates;
- (ii) identical paired initial tensors;
- (iii) FP32 training updates during the calibration phase that match the reference update to a relative tolerance of 2×10^{-5} ;
- (iv) bit-identical FP32 model outputs at updates spanning every TaperNorm gate state;

Table 4: Validity checks and actual forgetting F (full-dev web CE, nats/token). “Gap” is the absolute prefix web-CE difference relative to RMS (limit 2%). “Code gain” is the Python branch’s reduction in non-W code CE from the switch to $s = 6,104$ (minimum 0.05).

Seed	Prefix web CE			F , web branch		F , Python branch		Code gain (R / T)
	RMS	Taper	Gap	RMS	Taper	RMS	Taper	
101	4.5846	4.5716	0.28%	−0.128	−0.128	+1.655	+1.636	3.55 / 3.55
102	4.5795	4.5666	0.28%	−0.123	−0.122	+1.688	+1.668	3.60 / 3.46
103	4.5931	4.5773	0.34%	−0.128	−0.124	+1.655	+1.633	3.35 / 3.34

- (v) a vectorized evaluator that reproduces the reference evaluator’s CE and class sums on the real development data, to a relative tolerance of 10^{-9} ;
- (vi) identical decision categories on 3,000 randomly generated evidence sets;
- (vii) checkpoint resume whose deviation from an uninterrupted run is of the same order as run-to-run GPU nondeterminism.

The primary contrasts in Section 5 were also recomputed independently from the raw evaluation records, and they agree with the frozen analysis output.

5 Study 1: results

5.1 Validity checks

Table 4 shows that every guardrail passed.

- **Comparable prefixes.** The relative gap in prefix web CE was 0.28–0.34% (limit 2%), with Taper-minus slightly lower in every seed. Over the last five prefix evaluations, both conditions were still improving at nearly the same rate (mean slope -0.00244 vs. -0.00249 nats per million tokens).
- **Strong adaptation.** Both conditions adapted strongly to Python: non-W code CE fell by 3.34–3.60 nats (minimum 0.05), and the adaptation curves almost coincide (Figure 2c).
- **Large forgetting in both conditions.** The Python branch raised web CE by 1.63 to 1.69 nats/token relative to the switch, while web continuation lowered it by 0.12–0.13 (Figure 2a).

5.2 Primary endpoint

Table 5 gives the primary results. Mean $D = -0.0223$ nats/token (SD 0.0034; descriptive 95% interval $[-0.0308, -0.0139]$), and D is negative in all three seeds. Since $\bar{D} < 0.015$ and no early transient reached 0.05 (Section 5.3), the pre-specified decision is **stop: small observed effect** (rule 6). The data provide no evidence for H1. The close agreement of the three seeds should not be over-trusted: three fresh seeds in Study 2 gave web-to-Python contrasts of the opposite sign, none of them inside this interval (Section 6.5).

The other pre-specified quantities point the same way as D :

- G_{web} is close to zero (mean -0.0019), so the difference sits in the Python branch itself: the direct differential forgetting Q averages -0.0204 .

Table 5: Primary contrast D at $s = 6,104$ and secondary contrasts (nats/token; full development set). Positive values mean more excess web forgetting for Taper-minus. “Matched” is the matched-adaptation differential forgetting at the furthest target reached by every seed ($s = 6,104$).

Seed	D	$D(3050)$	$D(1525)$	G_{web}	Q	Matched ($s=6104$)
101	-0.0200	+0.0109	+0.0369	-0.0006	-0.0194	-0.0308
102	-0.0207	-0.0124	+0.0224	-0.0009	-0.0199	-0.0182
103	-0.0262	+0.0193	-0.0286	-0.0042	-0.0220	-0.0286
Mean	-0.0223	+0.0059	+0.0102	-0.0019	-0.0204	-0.0259
SD	0.0034	0.0165	0.0344	0.0020	0.0014	0.0067

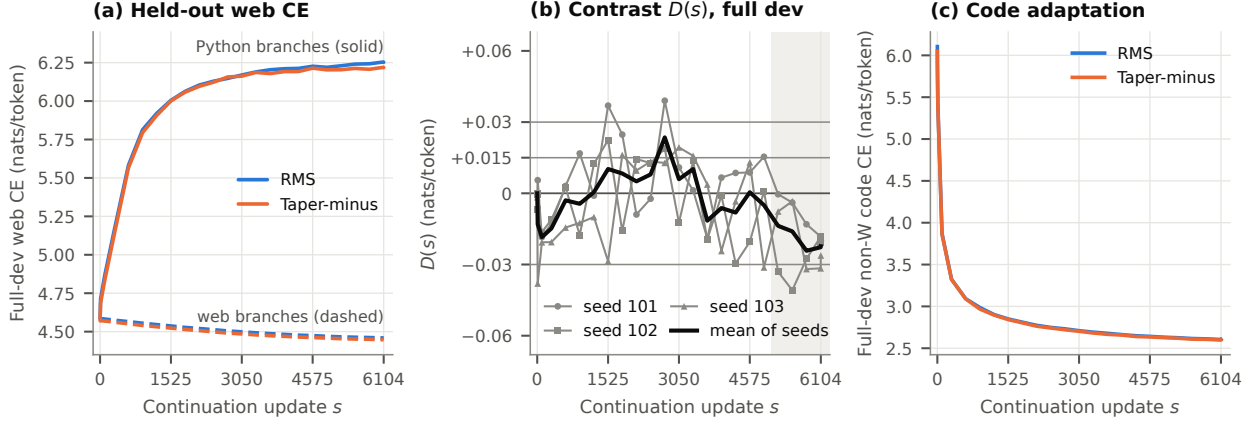


Figure 2: Continuation trajectories on the full development sets (means over three seeds unless stated; s is the continuation update). **(a)** Held-out web CE in all four branches. RMS and Taper-minus nearly coincide within each branch, and the Python branches show large forgetting. **(b)** The contrast $D(s)$ for each seed and their mean. The labelled horizontal lines are the pre-specified thresholds (+0.03 proceed, +0.015 small-effect bound, -0.03 opposite direction); the decision uses only $s = 6,104$ (and $s = 3,050$ for rule 4). The shaded band marks the late window ($s \geq 5,185$) used in the post hoc description of Section 5.3. **(c)** Non-whitespace code CE on the Python branch, showing near-identical adaptation in the two conditions.

- The matched-adaptation contrast at $s = 6,104$ averages -0.0259 , negative in every seed. On the quick development set, Taper-minus reached the RMS code-CE target before the corresponding RMS update in 9 of 9 seed-target pairs (Appendix B). At the furthest target, then, Taper-minus also showed slightly less web forgetting when compared at matched code adaptation.
- The per-document and per-class decompositions reconstruct D exactly. The effect is broad rather than driven by a few documents: median per-document D_i lies between -0.027 and -0.023 . By token class, alphanumeric tokens contribute negatively in all 3 seeds and give the largest share in 2 of them; in seed 101, punctuation and symbol tokens contribute more (Appendix B).
- Rare tokens (flag R, 5,147–5,285 labels per seed) have a more negative per-label contrast (mean D on R between -0.206 and -0.082), but because they are rare they contribute only -0.0005 to -0.0002 to D , at most 3% of it (Appendix B).

5.3 How the contrast developed during continuation

The difference between the conditions is small in absolute terms: $|\bar{D}|$ is 1.3% of the mean Python-branch forgetting, which is 1.66 nats/token. The decision uses only the endpoint, but the full-development contrast was measured at 25 points (Figure 2b). The windows below group those points after the fact; they describe the trajectory and were not part of the protocol.

- **Early** ($s = 10\text{--}305$): the seed-mean $D(s)$ was between -0.0185 and -0.0130 .
- **Middle** ($s = 610\text{--}4,880$, 15 points): the seed-mean $D(s)$ averaged $+0.002$ and ranged from -0.012 to $+0.024$, with no consistent sign.
- **Late** ($s = 5,185\text{--}6,104$, 5 points): the seed-mean $D(s)$ was negative at all 5 points and averaged -0.020 . Throughout this window the learning rate was within 9% of its final value.

In the late window $D(s)$ was negative in every seed at every point (15 of 15 seed-point values), and the across-seed SD fell from an average of 0.0164 over the middle window to 0.0034 at the endpoint, 4.8 times smaller. Seed agreement was not unique to the end, however: at $s = 100$ the seed-mean $D(s)$ was -0.0185 with an across-seed SD of 0.0021. The trajectory therefore shows two seed-consistent negative phases, early and late, separated by a noisy middle, and low seed spread is not specific to the low-learning-rate phase. The negative endpoint reflects the late phase, not a difference present throughout continuation. The pre-specified transient check was not triggered: the quick-dev seed-mean $D(s)$ peaked at $+0.015$ (at $s = 400$) within the first 1,000 updates, against a threshold of 0.05.

5.4 Exploratory analyses (not pre-specified)

The analyses in this subsection were chosen after the results were known. They are descriptive, and none of them enters the decision.

Activation scales at the normalizer inputs. The run logged forward-only probes of the activation h entering each of the 12 internal normalizers, on the last 256 training windows of each domain (131,072 positions per domain), at the switch and at the end of each branch. For each site we take the scale to be the root mean square of $\|h\|$ over the probe positions and work with natural-log ratios of scales, which treat growth and shrinkage symmetrically. For the absolute quantities ($|\cdot|$) we take the absolute log-ratio for each of the 36 site-seed pairs and then average; the signed quantities are plain averages, also shown as multiplicative factors. Table 6 gives three quantities:

- **The code-vs-web gap at the switch.** The mean absolute log-ratio was 0.097 (RMS) and 0.123 (Taper-minus): code and web activations differed in scale by roughly 10% and 13%.
- **The change in the web-probe scale during each branch.** In the RMS model the scale shrank, by a factor of 0.81 in the web branch (36 of 36 site-seed pairs shrank) and 0.82 in the Python branch (30 of 36). In Taper-minus it changed much less and mostly grew, by factors of 1.04 and 1.06 (mean absolute log-ratios 0.054 and 0.077).
- **The code-specific part of that change** (Python branch minus web branch, per site and seed): a mean absolute log-ratio of 0.042 (RMS) and 0.042 (Taper-minus), about 4% in both.

The largest scale change, the shrinkage in the RMS model, happened in both branches, so it was not caused by the code data. The code-specific change was small and the same size in both architectures. Study 2 turned these measures into a pre-specified manipulation check (Section 6.5).

Table 6: Exploratory: scale of the activations entering the 12 internal normalizers, from the logged diagnostic probes, as natural-log ratios of per-site scales. Rows marked $|\cdot|$ take the absolute log-ratio for each of the 36 site-seed pairs and then the mean; signed rows are plain means, with the corresponding multiplicative factor in parentheses.

Natural-log scale ratio (mean over 12 sites $\times$ 3 seeds)	RMS	Taper-minus
Code vs. web at the switch, $ \cdot $	0.097	0.123
Web-probe change, web branch, $ \cdot $	0.206	0.054
Web-probe change, Python branch, $ \cdot $	0.207	0.077
signed, web branch (factor)	$-0.206 (\times 0.81)$	$+0.040 (\times 1.04)$
signed, Python branch (factor)	$-0.196 (\times 0.82)$	$+0.060 (\times 1.06)$
site-seed pairs that shrank, web / Python branch	36 / 30 of 36	6 / 3 of 36
Code-specific change (Python minus web branch), $ \cdot $	0.042	0.042

Document bootstrap. To see how much of the uncertainty in D comes from the finite development set, we resampled the 4,409 web development documents with replacement (10,000 replicates; the same resampled documents for all arms and seeds) and recomputed the endpoint D . The 95% percentile interval for mean D is $[-0.0235, -0.0211]$, with a bootstrap standard error of 0.0006; per-seed standard errors are 0.0009–0.0010 (Appendix B). This interval treats the trained models as fixed, so it measures only evaluation-sample noise. Comparing like with like, the between-seed SD of D (0.0034) is about 3.6 times the per-seed bootstrap standard error, and the seed-based standard error of the mean (0.0020) is about 3.3 times the bootstrap standard error of the mean. Uncertainty about D is therefore dominated by training variation across seeds, and the bootstrap interval must not be read as uncertainty about the effect itself.

6 Study 2: a second domain, a replication and a manipulation check

Study 1 left two questions open. Its exploratory scale analysis suggested that the premise of H1 was weak for Python, so its null result says little about a shift that does move activation scales; and three seeds estimate seed-to-seed variation poorly. Study 2 is a separate study with its own protocol.² It adds no seeds to Study 1’s analysis and changes none of Study 1’s endpoints or decisions. It asks three questions.

- **H2 (primary).** After a switch from web text to a new domain X , chosen by a fixed rule, does Taper-minus show more persistent forgetting of held-out web text than RMS, relative to continuing web training? The estimand D_X is Equation (1) with the X branch in place of the Python branch, at $s = 6,104$, over seeds 101–106. As before, positive values mean that Taper-minus forgets more.
- **R2 (replication).** Does Study 1’s web-to-Python result replicate on three fresh seeds (104–106), with Study 1’s estimand, guardrails and decision rules unchanged?
- **M (manipulation check, descriptive).** Was the scale premise in fact stronger for X than for Python? M applies the measures of Section 5.4, now pre-specified, to both domains in the

²The frozen protocol calls itself “pre-registered” (for example, in the heading of its Section 9.4) and calls R2 “confirmatory”. That wording was written before Amendment 1 deferred the public registry entry, and the frozen text cannot be changed. In this paper Study 2 is described as specified in advance and hash-timestamped (Section 6.3), and R2 as a replication within a screening study.

Table 7: Study 2 runs. Every run is done for both conditions, paired as in Study 1 (Figure 1).

Seeds	Prefix	Web branch	Python branch	X branch
101–103	Study 1 (reused)	Study 1 (reused)	Study 1 (pooled estimate only)	new
104–106	new	new	new	new

same models.

Following the protocol, we write X for the selected domain. To avoid a clash of names, Study 1’s token class of special, control and invalid-fragment tokens is called C throughout this paper.

6.1 Design

The model, conditions, optimizer, learning-rate schedule, precision, evaluation schedule and analysis are those of Study 1. The code was copied from Study 1’s runner, and a test suite checks every copied function against the original. Table 7 shows the runs. Seeds 101–103 reuse Study 1’s six saved switch states: only their X branches are new, and these are compared with Study 1’s own web-branch records. Seeds 104–106 are new throughout. Each of these seeds trains a prefix and then three branches (web, Python and X), all of which restore the identical saved switch state.

Lineage check. Seeds 101–103 compare an X branch trained in a new session with web-branch records from an earlier one. Every branch therefore begins, before its first update, by re-evaluating its restored switch state and comparing the web cross-entropies and the per-site activation energies with the records of that same state (Study 1’s records for seeds 101–103, the run’s own for 104–106). A difference above 10^{-3} (absolute for cross-entropy, relative for energies) stops the worker and makes the decisions invalid.

Decision rules and readings. H2 applies the rules of Table 1 unchanged to D_X over six seeds. Adaptation is the drop in the X development set’s non-W cross-entropy over the X branch. R2 applies Table 1 exactly as Study 1 did. Study 2 adds the following, all fixed in its protocol:

- 95% t -intervals with multipliers 2.571 (six seeds) and 4.303 (three seeds);
- an *equivalence reading*: if the 90% t -interval lies entirely inside $(-0.015, +0.015)$, the report states “equivalent within ± 0.015 ” (two one-sided t -tests at $\alpha = 0.05$). Under the protocol this reading is the only basis on which Study 2 may say that an effect is no larger than 0.015 nats/token;
- descriptive outputs: H2 on the fresh seeds 104–106 alone, since the switch states of seeds 101–103 also served the domain selection; a pooled six-seed web-to-Python estimate that includes Study 1’s published seeds; a document bootstrap; and per-phase gradient clipping;
- the manipulation check and its two readings. The premise is *stronger for X at the switch* if the mean absolute switch-time gap of X exceeds that of Python in both conditions, and *stronger for X in training* if the mean absolute domain-specific change does;
- an interpretation matrix (Table 8) that fixes how H2’s category is read together with the in-training reading of the manipulation check.

Table 8: Pre-specified interpretation matrix of Study 2 (protocol, Section 9.4). “Premise stronger” is the in-training reading of the manipulation check. The reading that applies to the outcome is in bold; its second sentence, in grey, needs the equivalence reading, which was not met.

H2 category	Premise stronger	Permitted reading
Proceed	yes	TaperNorm shows persistent excess forgetting where the scale premise is present (screening level, this setting)
Proceed	no	Excess forgetting on X , not explained by the measured scale premise
Stop, opposite or transient	yes	Evidence against the scale-mismatch account of H1 in this setting. (← outcome) With the equivalence reading, also “no excess larger than 0.015” (not met).
Stop, opposite or transient	no	A second domain pair without excess forgetting; uninformative about the mechanism, because the premise was again weak
Inconclusive, limited or invalid	any	No conclusion about H2

Table 9: Domain selection: the statistic S for each candidate, overall, per condition and on each half of the probe sample (windows 1–128 and 129–256). Python is computed the same way on Study 1’s Python probe, as the reference.

Candidate	S (both)	RMS	Taper-minus	First half	Second half
Chinese web text (mC4 zh) (selected)	0.0934	0.0757	0.1111	0.0931	0.0938
Russian web text (mC4 ru)	0.0893	0.0732	0.1054	0.0887	0.0898
German web text (mC4 de)	0.0514	0.0469	0.0558	0.0511	0.0518
Mathematical web text (OpenWebMath)	0.0119	0.0126	0.0112	0.0116	0.0128
Python (pilot’s code, reference)	0.1099	0.0971	0.1228		

6.2 Selecting the domain

The protocol fixed four candidates, each pinned to a dataset revision: German, Russian and Chinese web text from the multilingual C4 corpus (mC4) (Xue et al., 2021), at the same revision as Study 1’s C4, and mathematical web text from OpenWebMath (Paster et al., 2024). Other programming languages were not candidates, since Study 1 had found the premise weak for code. For each candidate, a probe sample of 256 windows from its training side (131,072 positions, the size of Study 1’s probes) was passed forward through Study 1’s six switch states. The selection statistic is

$$S(c) = \text{mean} \left| \frac{1}{2} \ln (E_c / E_{\text{web}}) \right|, \quad (4)$$

where E is the mean squared norm of the activation entering a normalizer and the mean runs over 12 sites, three seeds and both conditions. This is the switch-time gap of Section 5.4. The candidate with the largest S becomes X . Before selecting, the probe had to reproduce Study 1’s recorded switch-time energies for web and Python within a relative 10^{-3} ; the reproduction was exact (largest relative difference 0.0).

Table 9 gives the result. Chinese web text was selected ($S = 0.0934$), narrowly ahead of Russian, and S was stable across the two halves of every probe sample. No candidate exceeded Python’s $S = 0.1099$. The protocol anticipated this case: Study 2 proceeds, and the report of H2 must state that the selection did not find a domain with a stronger switch-time premise than Python. A

forward probe at the switch cannot see how activation scales change during training, which is what the manipulation check measures afterwards.

Data. The Chinese data were prepared with Study 1’s pipeline (GPT-2 tokenization, packing, and exact and MinHash near-duplicate removal), with two additions. Documents that exactly duplicated any of Study 1’s documents, or nearly duplicated one of its development documents, were removed; and the audit also compared retained Chinese documents with Study 1’s development documents. Reading 2 training shards and 1 validation shard (126.5M raw training tokens), deduplication removed 698 near duplicates and 0 exact duplicates; 0 documents duplicated Study 1’s corpus. The audit found no pair at or above 0.85, among 2,443 cross-split pairs or among 2,000 pairs with Study 1’s development documents. The training split holds 110,000,129 tokens (56,276 documents) and the development split 2,097,152 labels (1,139 documents); a reserved test split (4,201 documents) was sealed and never used. Study 1’s data orders for seeds 101–103 were regenerated bit for bit before the new orders were written.

GPT-2’s byte-level BPE (Radford et al., 2019) was trained mostly on English, and it splits most Chinese characters into byte fragments that are not valid UTF-8 on their own. 85% of the Chinese development labels therefore fall in class C, against 2.2% for web text. The Chinese shift is as much a shift in which tokens the model must predict as a shift in content.

6.3 Pre-specification and timeline

The protocol was written after Study 1’s results and paper were known; its Section 12.2 lists everything that was known or inspected beforehand. No activation of any candidate domain had been measured, and no Study 2 job had run. The author then decided to execute Study 2 before filing a public registry entry. Amendment 1, made before any job, records that decision and adds the measures below. A registration manifest (`study2/registration-manifest.json`, SHA-256 `3eeda992819a...`) lists the SHA-256 of the protocol, including the amendment, and of every Study 2 code file. All times are UTC.³

- 21:55 UTC on 8 October: the manifest’s SHA-256 is submitted to four OpenTimestamps calendar servers. Only the hash leaves the author’s computer.
- 21:56 UTC on 8 October: the first Study 2 job, the domain probe, is created on Kaggle. Every Study 2 notebook embeds the manifest byte for byte, and Kaggle records each version’s creation time on its own servers. The probe, the data preparation and every training session also write the manifest’s SHA-256 into their outputs.
- 22:59 UTC on 8 October: the main run starts.
- 01:59 UTC on 9 October: Bitcoin block 970,569 is mined, carrying the timestamp attestation of the manifest.

The Bitcoin attestation is the only one of these records that depends on neither the author nor a single service provider, but it is also the weakest bound. Block times are accurate only to about two hours, and by the time of this block the *X* branches of seeds 101–103 had already reached their endpoints inside the running session. The time of submission to the calendars comes from the author’s machine; the calendar operators record it, but the proof does not establish it. That the plan was fixed before any outcome existed therefore rests on Kaggle’s server-side version times: the notebook versions that embed the manifest were created between 21:56 UTC on 8 October and

³The protocol’s own dates (8 and 9 October 2026) are the author’s local dates, UTC+3.

the start of the main run. (The times given here were printed by the author’s machine at each push; Kaggle’s version history records them independently.) Three further facts limit what could have changed after the run started. The decisions were computed in-process by the runner, its decision thresholds are constants in its source, and every session recorded a runner SHA-256 equal to the registered one. Amendment 1 defers the public registry entry until after execution, so that entry cannot establish pre-specification on its own. Study 2 should be read as *specified in advance and hash-timestamped before any run, with public registration deferred until after execution*, not as conventionally pre-registered.

6.4 Execution and implementation checks

Before the main run, a smoke test exercised the whole protocol end to end on toy models (with interruptions and resumes), checked the lineage of Study 1’s six switch states, and measured throughput on seed 104’s prefix only; it produced no outcome data. The main run then used two sessions on two T4 GPUs, of 11.12 and 4.06 hours (15.17 hours in all), with PyTorch 2.11.0 in the Kaggle image of Study 1’s run.

- All 12 condition–seed runs completed. There were no restarts, missing records, failures or integrity problems, and there were 0 FP16 overflow retries.
- All 24 lineage checks were bit-exact: every restored switch state re-evaluated to exactly its recorded cross-entropies and energies, including Study 1’s six switch states in a new session.
- Training took 0.37 s per update in each condition. Gradient clipping was active in 99.5% (RMS) and 99.7% (Taper-minus) of Chinese-branch updates, against about 90% and 94% on the web and Python branches of the RMS models of the fresh seeds (Appendix C).
- The decision report written by the runner was recomputed independently from the downloaded records, and every contrast, interval and manipulation-check value agrees with it (`paper/make_assets.py` asserts this on every build).

6.5 Results

Guardrails and forgetting. The relative prefix gap was 0.23–0.34% (limit 2%), and both conditions adapted strongly to Chinese: non-W Chinese cross-entropy fell by 2.18–2.33 nats (minimum 0.05; Figure 3c). The Chinese branch was nearly catastrophic for web text. Mean web cross-entropy rose from 4.58 at the switch to 6.99, a forgetting F of 2.34 to 2.55 nats/token, against 1.63–1.69 for Python in Study 1; web continuation lowered it to 4.46 (Figure 3a).

H2. Table 10 gives the results. Mean $D_X = -0.0325$ nats/token (SD 0.0341; 95% interval $[-0.0683, +0.0032]$), negative in 5 of 6 seeds and positive in seed 102 ($+0.0065$). Because the mean is at or below -0.03 , the pre-specified category is **opposite direction** (rule 3): Taper-minus forgot less web text than RMS after the switch to Chinese. The 90% interval, $[-0.0606, -0.0045]$, does not lie within ± 0.015 , so equivalence is not shown, and the protocol’s own reading does not allow the statement that the effect is no larger than 0.015 (a post hoc one-sided reading follows the replication below).

The category label needs three qualifications.

- **The margin is narrow.** The mean lies just past the threshold, and the 95% interval includes zero.

Table 10: H2: the contrast D_X at $s = 6,104$ and secondary contrasts for web $\rightarrow$ Chinese (nats/token; full development set; positive means more excess web forgetting for Taper-minus). $\dagger$ The switch state is Study 1’s. “Matched” is the matched-adaptation differential forgetting at the furthest target reached by every seed. “X gain” is the drop in non-W Chinese cross-entropy over the branch.

Seed	D_X	$D_X(3050)$	$D_X(1525)$	G_{web}	Q	Matched ($s=6104$)	Gap	X gain (R / T)
101 $\dagger$	−0.0886	−0.0260	+0.0452	−0.0006	−0.0880	−0.1137	0.28%	2.30 / 2.33
102 $\dagger$	+0.0065	−0.0176	+0.0012	−0.0009	+0.0074	−0.0466	0.28%	2.31 / 2.33
103 $\dagger$	−0.0351	−0.0739	−0.1081	−0.0042	−0.0309	−0.0854	0.34%	2.23 / 2.27
104	−0.0513	+0.0071	−0.0405	−0.0026	−0.0487	+0.0460	0.26%	2.21 / 2.24
105	−0.0179	−0.0070	+0.0399	−0.0026	−0.0153	−0.0130	0.23%	2.18 / 2.19
106	−0.0089	−0.0856	−0.0350	−0.0006	−0.0082	−0.0143	0.30%	2.28 / 2.31
Mean	−0.0325	−0.0338	−0.0162	−0.0019	−0.0306	−0.0378		
SD	0.0341	0.0374	0.0576	0.0015	0.0340	0.0571		

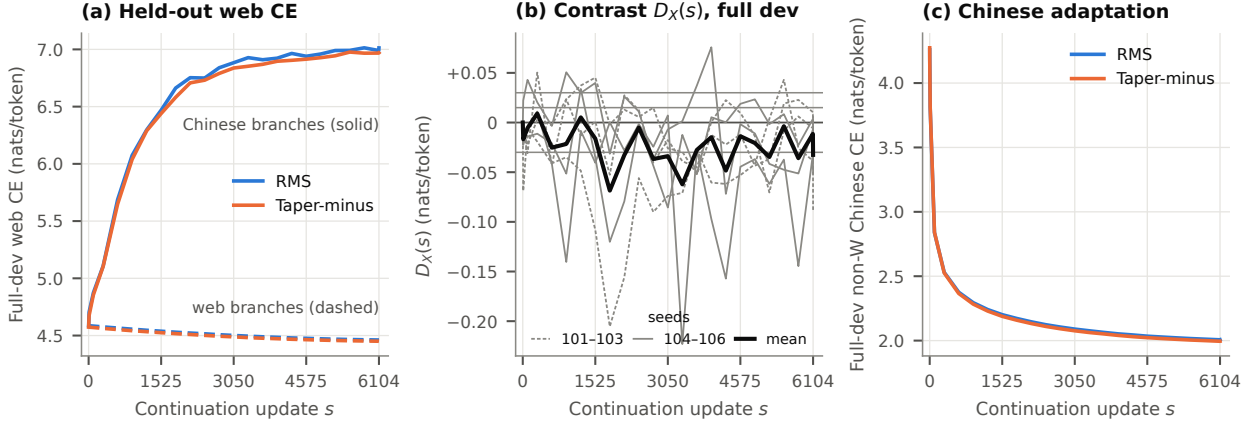


Figure 3: Study 2, web $\rightarrow$ Chinese, on the full development sets (means over six seeds unless stated). (a) Held-out web cross-entropy on the Chinese and web branches. (b) The contrast $D_X(s)$ for each seed (dashed: seeds that reuse Study 1’s switch states) and the mean. The horizontal lines are the pre-specified thresholds (+0.03, +0.015, −0.03); the decision uses only $s = 6,104$ (and $s = 3,050$ for rule 4). (c) Non-W cross-entropy on the Chinese development set during the Chinese branch.

- **It depends on single seeds.** On the fresh seeds alone the mean is -0.0260 (95% interval $[-0.0816, +0.0296]$), which does not reach -0.03 . The most negative seed is 101 (-0.0886), whose switch state is Study 1’s and also served the domain selection. Post hoc, leaving out one seed at a time gives means between -0.040 and -0.021 , and leaving out either of seeds 101 and 104 raises the mean above -0.03 , which would make the category *stop*.
- **Seed variation is large.** The between-seed SD of D_X (0.0341) is about 10 times Study 1’s SD of D (0.0034), which the protocol had used for planning. Individual trajectories are also noisy: a seed’s $D_X(s)$ changed by 0.040 on average between adjacent full evaluations, 305 updates apart (post hoc; Figure 3b), more than the 0.03 that separates zero from either outer threshold.

The other pre-specified quantities point the same way as D_X :

- $D_X(3050)$ averages -0.0338 . G_{web} is close to zero (-0.0019), so the difference lies in the Chinese branch itself: Q averages -0.0306 . The matched-adaptation contrast at $s = 6,104$ averages -0.0378 .

Table 11: R2: the web $\rightarrow$ Python contrast D on the fresh seeds, with Study 1’s estimand and rules (nats/token; full development set). “Code gain” is the drop in non-W Python cross-entropy from the switch. Study 1’s seeds 101–103 are in Table 5.

Seed	D	$D(3050)$	G_{web}	Q	Matched ($s=6104$)	Gap	Code gain (R / T)
104	+0.0160	+0.0224	−0.0026	+0.0187	+0.0271	0.26%	3.50 / 3.58
105	+0.0102	+0.0197	−0.0026	+0.0128	+0.0181	0.23%	3.42 / 3.44
106	+0.0018	−0.0295	−0.0006	+0.0024	+0.0068	0.30%	3.38 / 3.64
Mean	+0.0093	+0.0042	−0.0019	+0.0113	+0.0174		
SD	0.0071	0.0292	0.0012	0.0082	0.0102		

- The transient rule was not triggered: the quick-dev seed-mean $D_X(s)$ peaked at +0.012 (at $s = 650$) within the first 1,000 updates, against a threshold of 0.05. The full-dev seed-mean $D_X(s)$ was negative at 22 of the 24 points after the switch. Over Study 1’s post hoc late window ($s \geq 5,185$) it averaged −0.0237 and was negative at 5 of 5 points.
- $|\bar{D}_X|$ is 1.3% of the mean Chinese-branch forgetting (2.41 nats/token).
- The effect is broad rather than driven by a few documents: the median per-document D_i lies between −0.064 and +0.010 and is negative in 5 seeds. Class P contributes negatively in 5 seeds and class A in 4; rare tokens contribute at most 0.0007 in magnitude (Appendix C).
- The document bootstrap gives a 95% interval of $[-0.0339, -0.0311]$ for mean D_X , far narrower than the seed-based interval: as in Study 1, uncertainty comes from training variation, not from the evaluation sample.

R2. Table 11 gives the replication. The guardrails passed (prefix gaps 0.23–0.30%, Python adaptation 3.38–3.64 nats). Mean $D = +0.0093$ (SD 0.0071; 95% interval $[-0.0084, +0.0271]$), so the category is **stop: small observed effect**, and Study 1’s decision replicates. Equivalence is again not shown (90% interval $[-0.0027, +0.0214]$). The decision replicates, but the sign does not. D was positive in every fresh seed, where it had been negative in every seed of Study 1, and none of the fresh values lies inside Study 1’s descriptive 95% interval $[-0.0308, -0.0139]$. The sign split coincides with the session split: all three negative contrasts come from Study 1’s run and all three positive ones from Study 2’s. If seeds are exchangeable, a split this clean has probability 0.1. We checked for a systematic difference between the two groups and found none (post hoc). Across both conditions, prefix web cross-entropy was 4.567–4.593 in Study 1’s seeds and 4.571–4.592 in the fresh ones; web-branch forgetting was −0.128 to −0.122 and −0.126 to −0.120; and Python-branch forgetting was 1.633–1.688 and 1.627–1.667. Both sessions used the same pinned software image, and Study 2 re-evaluated Study 1’s switch states bit-exactly. Three seeds had understated the seed-to-seed variation: the SD of the six web-to-Python values is 0.0180, about 5 times Study 1’s. Pooled over all six seeds (descriptive only, and including Study 1’s published seeds), mean $D = -0.0065$ with a 95% interval of $[-0.0254, +0.0125]$. Table 12 collects every interval of both studies’ contrasts.

A post hoc one-sided reading. H1 predicts an excess, so “no evidence for H1” rests on a one-sided question: is an excess of +0.015 ruled out? The protocol allows the statement “no larger than 0.015” only through its two-sided equivalence reading, so what follows is a post hoc non-inferiority

Table 12: Means, SDs and descriptive intervals of the endpoint contrasts (nats/token). The last column is the pre-specified equivalence reading of Study 2: whether the 90% interval lies inside $(-0.015, +0.015)$. Study 1 did not pre-specify this reading; its row is shown for comparison.

Contrast	Seeds	Mean	SD	95% t -interval	90% t -interval	Equivalent
Chinese, D_X (H2)	101–106	−0.0325	0.0341	[−0.0683, +0.0032]	[−0.0606, −0.0045]	no
fresh seeds only	104–106	−0.0260	0.0224	[−0.0816, +0.0296]	[−0.0637, +0.0117]	no
Python, D (R2)	104–106	+0.0093	0.0071	[−0.0084, +0.0271]	[−0.0027, +0.0214]	no
pilot (Study 1)	101–103	−0.0223	0.0034	[−0.0308, −0.0139]	[−0.0280, −0.0166]	no
pooled (descriptive)	101–106	−0.0065	0.0180	[−0.0254, +0.0125]	[−0.0213, +0.0084]	no

reading, not a pre-specified one. The one-sided 95% upper bound of a mean is the upper end of its 90% interval in Table 12.

- **Chinese (six seeds):** −0.0045. This rules out an excess of +0.015. Equivalence failed only on the lower side, where Taper-minus may forget less.
- **Python, fresh seeds alone:** +0.0214. This does not rule out an excess of +0.015.
- **Python, six seeds pooled** (descriptive, including Study 1’s published seeds): +0.0084. This rules it out.

The Chinese bound lies below zero, yet we do not use the same one-sided logic to claim protection. The asymmetry is deliberate. H1 is directional, so a one-sided test against the excess it predicts fits it. Protection was never hypothesized, so it gets the default two-sided reading, whose 95% interval includes zero; and the opposite-direction label depends on single seeds.

Manipulation check. Table 13 gives the pre-specified scale measures for both domains in the same six seeds.

- **At the switch,** Chinese activations differed from web activations no more than Python activations did (mean absolute log-ratios 0.085 and 0.122 for Chinese, 0.093 and 0.136 for Python, RMS and Taper-minus), consistent with the selection probe. Premise stronger for X at the switch: *no*.
- **During training,** the domain-specific change of the web-probe scale was 0.165 (RMS) and 0.183 (Taper-minus) for Chinese, against 0.039 and 0.042 for Python: 4.3–4.4 times larger. Premise stronger for X in training: *yes*.
- **The architectures moved in opposite directions.** On the Chinese branch the web-probe scale shrank by a factor of 0.72 in the RMS models and grew by a factor of 1.21 in the Taper-minus models (web branch: 0.82 and 1.05).

The in-training measure is taken after treatment, in models that had by then forgotten different amounts: the Chinese branch forgot more than the Python branch, and most Chinese labels are byte fragments that mainly load the tied embedding and output layer. A larger domain-specific scale change may therefore be partly a symptom of heavier forgetting rather than an independent premise, and the check cannot separate the two. The measure taken before treatment, the switch-time gap, was not stronger for Chinese.

Table 13: Manipulation check (pre-specified, descriptive): scales of the activations entering the 12 internal normalizers, as natural-log ratios, for Chinese (X) and Python in the same six seeds. “Switch gap” compares the domain’s probe with the web probe at the switch. The branch rows give the change of the web-probe scale from the switch to the end of the web branch or of the shifted (Chinese or Python) branch, so the web-branch row is the same for both domains. “Domain-specific change” is the shifted-branch change minus the web-branch change, per site and seed (bold: the in-training premise). Rows marked $|\cdot|$ average the absolute log-ratio over 12 sites $\times$ 6 seeds; the signed row is a plain mean, with its multiplicative factor.

Log-ratio of scales	Chinese (X)		Python	
	RMS	Taper-minus	RMS	Taper-minus
Switch gap, domain vs. web, $ \cdot $	0.085	0.122	0.093	0.136
Web branch: web-probe change, $ \cdot $	0.203	0.059	0.203	0.059
Shifted branch: web-probe change, $ \cdot $	0.348	0.225	0.204	0.075
signed (factor)	$-0.322 (\times 0.72)$	$+0.191 (\times 1.21)$	$-0.196 (\times 0.82)$	$+0.056 (\times 1.06)$
Domain-specific change, $ \cdot $	0.165	0.183	0.039	0.042

Pre-specified reading. H2’s category is opposite direction and the premise was stronger for X in training, so the matrix (Table 8) permits the reading *evidence against the scale-mismatch account of H1 in this setting*. Without the equivalence reading, its second sentence does not apply. The reading is sturdier than the label. Every leave-one-out mean stays below $+0.015$, so dropping any one seed would leave the category in the same row of the matrix, which groups stop, opposite and transient, and the permitted reading would be unchanged; only the label *opposite direction* depends on single seeds. The reading inherits the limits of the manipulation check, which is measured after treatment. Because of the qualifications above, we read the opposite-direction category as no evidence for H1 on Chinese, and not as evidence that TaperNorm protects against forgetting.

7 Discussion

What the two studies show. Over two domain shifts and twelve seed-level contrasts, replacing internal RMSNorm with TaperNorm did not produce the excess forgetting that H1 predicts. Neither shift reached the pre-specified screen for a meaningful excess. For Python, Study 1’s mean was negative and the fresh seeds’ mean, although positive, stayed below 0.015. For Chinese, the mean was negative and fell just into the opposite-direction category. How firmly an excess of $+0.015$ is ruled out differs, by the post hoc one-sided reading of Section 6.5: firmly for Chinese, for Python only when the six seeds are pooled, and not by the three fresh Python seeds alone. Study 2 also measured the premise with a pre-specified in-training check. Beyond what continued web training did, the Chinese shift changed activation scales during training about four times as much as Python did, and the two architectures responded differently: the RMS models’ scale shrank and the Taper-minus models’ grew. Taper-minus nevertheless did not forget more. Under Study 2’s pre-specified interpretation matrix this is evidence against the scale-mismatch account of H1 in this setting, subject to the limits of a check measured after treatment.

What they do not show. *Stop* is not evidence of equivalence (Lakens, 2017), and neither study met its equivalence reading. Nor is the opposite-direction category good evidence that TaperNorm protects against forgetting. The Chinese mean lay just past the threshold, its 95% interval includes zero, leaving out either of two single seeds would change the label to *stop*, and a single seed’s endpoint is noisy. The Chinese shift was also nearly catastrophic for web text in both architectures

(Figure 3a), a regime in which the difference between them is a small fraction of the total forgetting. In Study 1, apart from a brief early dip, the negative values of D appeared only in the final, low-learning-rate phase, and the training regime, including learning-rate decay, is known to affect forgetting (Mirzadeh et al., 2020). All intervals are conditional on one prepared corpus sample per domain and one training recipe; they are not population bounds.

Seed variation. The most consequential lesson for a design like ours concerns variance. Study 1’s three seeds agreed closely (SD 0.0034), which made its descriptive interval narrow, and Study 2’s protocol used that SD for planning. The fresh seeds then gave web-to-Python contrasts of the opposite sign, outside Study 1’s interval, and the Chinese contrasts varied about ten times as much as Study 1’s had. Seed-to-seed variation in neural-network training can be large (Dodge et al., 2020), and three seeds can understate it badly. Pairing on initialization and data order removes variation that the two conditions share, but not the variation that builds up differently in each of them over 15,260 updates. Future designs of this kind need many more seeds than we used, or an endpoint that averages several late evaluations rather than relying on one.

The premise and the mechanism. Study 1’s exploratory analysis found the scale premise weak for Python: code and web activations differed in scale by only about 10–13% at the switch, and the largest change during continuation, a shrinkage in the RMS model, happened to a similar extent when training simply continued on web text. Study 2’s in-training check found a premise about four times stronger for Chinese and still no excess forgetting, which makes it less likely that H1 failed only because the premise was absent. That check, however, is measured after treatment and cannot separate cause from symptom: a larger domain-specific scale change may partly reflect the heavier forgetting of the Chinese branch, and the measure taken before treatment, the switch-time gap, was not stronger for Chinese. We tested no mechanism, but several features of the setting may matter:

- Taper-minus keeps the final RMSNorm, which Kanavalau et al. (2026) identify as the essential anchor of scale.
- $\tilde{\gamma}$ and all weight matrices remain trainable after the taper, so a tapered model can absorb a change of scale in its weights.
- RMSNorm makes each sub-layer’s output invariant to the scale of the residual stream it reads, so weight decay can shrink the weights that set that scale without changing the function (van Laarhoven, 2017; Zhang et al., 2019a). This would explain the shrinkage of the RMS models’ scale in both studies and the opposite movement in Taper-minus, whose internal layers depend on that scale once its gate is zero.
- With GPT-2’s English-centred tokenizer, 85% of the Chinese labels are byte fragments. Much of the Chinese-branch forgetting may therefore come from the tied embedding and output layer, which the internal normalizers do not touch, rather than from the representations inside the network.

Testing these explanations would need targeted ablations, which neither study ran.

Specifying a cloud study in advance. Study 2 shows that a study run on free cloud GPUs can be fixed before it runs without a registry entry: hash the frozen protocol and code, timestamp the hash, embed the manifest in every cloud job, and check every reused model state bit-exactly against its own records before it trains. It also shows the weak point. The independent Bitcoin anchor

arrived about 3 hours after the main run started, so the earlier bound rests on the cloud provider’s records. Waiting a few hours for the attestation before starting the run, or filing a registry entry first, would remove that dependence at little cost.

8 Limitations

- **Scale.** The models have 17,718,784 parameters, about 73% of them in embeddings, and see 150M prefix tokens. Embedding drift may mediate or interact with the condition effect, and results may not transfer to larger models or longer training.
- **Breadth.** There are two domain pairs, one data sample per domain, one optimizer recipe, and one hardware and precision setting (FP16 on T4, not Study 1’s originally planned BF16). The auxiliary-loss variant of TaperNorm (Taper-plus) was not run.
- **Statistics.** Seeds are few, and the realized seed-to-seed variation was far larger than Study 1’s three seeds suggested and than Study 2’s planning assumed. All intervals are descriptive t -intervals, conditional on one prepared corpus per domain. Every measurement uses development data; the reserved test sets of both studies remain sealed.
- **The selected domain.** The selection statistic is a switch-time proxy, and it did not find a domain with a larger switch-time gap than Python. Chinese text under GPT-2’s tokenizer mixes a change of content with a change of the predicted token distribution, and it destroyed most of the models’ web performance in both architectures. Another domain, or a tokenizer that covers Chinese, might behave differently.
- **The manipulation check.** Its in-training measure is taken after treatment, in models that had forgotten different amounts, so it cannot separate a premise that causes forgetting from a symptom of it. The measure taken before treatment, the switch-time gap, was not stronger for Chinese.
- **Reuse across sessions.** For seeds 101–103, Study 2’s Chinese branches are compared with web branches trained in Study 1’s session. The bit-exact lineage checks and the pinned software image bound the differences between sessions but cannot exclude all of them. The same seeds’ switch states also served the domain selection.
- **Pre-specification.** Neither study was lodged with a registry before it ran. For Study 1, the earliest public record of the protocol and analysis code, a git commit, postdates both the start of the primary run and the appearance of seed 101’s endpoint values in the run’s live log (Section 3.6); the evidence that its analysis was fixed in advance is the configuration hash logged before any outcome and the unchanged runner hash. For Study 2, the Bitcoin attestation postdates the Chinese endpoints of seeds 101–103, and the earlier bound rests on Kaggle’s server-side records (Section 6.3). Git commit dates are set by the author’s machine and are not independent timestamps.
- **Thresholds.** The decision thresholds were set as investment thresholds without a power analysis, and Study 2’s equivalence bound reuses the smaller of them. The one-sided readings and the leave-one-seed-out analysis were added after the results were known.
- **Data audit.** The near-duplicate search is approximate. Its analytic recall is high for pairs above the similarity threshold, but the random-pair audits are only sanity checks with negligible power, repository names are an incomplete proxy for forks, and near duplicates between the Chinese data and Study 1’s training arrays were not searched.

- **Secondary and exploratory analyses.** Matched adaptation interpolates between evaluated checkpoints and describes curves; it does not evaluate an actual model. The trajectory windows, Study 1’s scale analysis, its document bootstrap and the trajectory noise in Study 2 were examined after the results were known. The scale measures use the last 256 training windows of each domain, which the models may have seen, and they describe mean scales over tokens, whereas RMSNorm also normalizes variation between individual tokens. Attention-entropy and gain diagnostics were logged but are not analysed here.

9 Conclusion

Two small studies, each specified before it ran, found no evidence that TaperNorm increases persistent forgetting after a domain shift. For web to Python, the pre-specified decision was *stop* in the pilot and again on three fresh seeds, although the fresh seeds’ contrasts were positive where the pilot’s had been negative. For web to Chinese, where a pre-specified in-training measure of the premise was about four times larger, mean D_X was -0.0325 nats/token, just past the threshold for the opposite direction, with a 95% interval that includes zero; a post hoc one-sided bound rules out an excess of $+0.015$. Neither study shows equivalence, and seed-to-seed variation was much larger than the pilot suggested. A more decisive test would need larger models, many more seeds, a tokenizer suited to the new domain, and an endpoint that averages several late evaluations.

Code and data availability

The following are available in the project repository:

<https://github.com/DanielDotComCoder/domain-shift-forgetting>

- **Study 1:** the code for data preparation, training, evaluation and the frozen analysis (`kaggle_h1/`); the frozen configuration, with data and source hashes, and the decision report; every evaluation record (per-document and per-class sufficient statistics, and the diagnostic probes); and the per-update training logs (`reports/h1-kaggle/`).
- **Study 2:** the protocol with Amendment 1, the registration manifest, its OpenTimestamps proof, the timestamp evidence and the execution log (`study2/`); the domain probe, data preparation, runner, notebook builder and tests (`kaggle_s2/`); and the selection record, the data manifest, audit and acceptance records, the smoke-test results, every evaluation and training record, the session records and the decision reports (`reports/s2-kaggle/`).

The script `paper/make_assets.py` regenerates every number, table and figure in this paper from those records. On every build it also re-checks the integrity chain: the configuration hashes, the registered protocol and runner hashes, the lineage records and, when the OpenTimestamps library is installed, the timestamp proof. The Study 2 domain-probe, smoke-test and main-run notebooks are public on Kaggle (user `danny00`), so the times of their versions can be checked. The data-preparation notebook stays private because its output contains the sealed reserved test split. Three things are not in the repository:

- the model checkpoints (switch states and branch-final weights, several gigabytes);
- the prepared token arrays, which can be regenerated from the pinned dataset revisions with the released preparation code;
- any corpus text. C4, mC4, OpenWebMath and The Stack are subject to their own licenses and terms of use.

Funding and affiliation

The author carried out this work as an independent researcher, without funding and without the involvement of any institution. All training ran on Kaggle’s free GPU quota.

Use of AI assistance

AI systems, mainly Claude (Anthropic) used as a coding agent, were used extensively in both studies. They drafted and revised the protocols, including Study 2’s protocol and its Amendment 1; wrote the data-preparation, training, orchestration and analysis code and its tests; submitted and monitored the Kaggle jobs through the Kaggle API, downloaded and checked their outputs, and created the OpenTimestamps proof; and drafted this manuscript, including the script that generates its numbers, tables and figures. AI systems were also used to review drafts of the manuscript. The author chose the research question and directed the work. The author also made every decision the protocols record as the author’s, including approving each study design and every amendment, accepting Study 1’s decision to stop, and running Study 2 before filing a registry entry. The author reviewed the results and this paper and takes full responsibility for its content, including any errors.

References

- Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- Andrew Brock, Soham De, Samuel L. Smith, and Karen Simonyan. High-performance large-scale image recognition without normalization. In *Proceedings of the 38th International Conference on Machine Learning*, 2021.
- Andrei Z. Broder. On the resemblance and containment of documents. In *Proceedings of Compression and Complexity of SEQUENCES 1997*, pages 21–29, 1997.
- Jesse Dodge, Gabriel Ilharco, Roy Schwartz, Ali Farhadi, Hannaneh Hajishirzi, and Noah Smith. Fine-tuning pretrained language models: Weight initializations, data orders, and early stopping. *arXiv preprint arXiv:2002.06305*, 2020.
- Shibhansh Dohare, J. Fernando Hernandez-Garcia, Qingfeng Lan, Parash Rahman, A. Rupam Mahmood, and Richard S. Sutton. Loss of plasticity in deep continual learning. *Nature*, 632:768–774, 2024.
- Robert M. French. Catastrophic forgetting in connectionist networks. *Trends in Cognitive Sciences*, 3(4): 128–135, 1999.
- Kshitij Gupta, Benjamin Thérien, Adam Ibrahim, Mats L. Richter, Quentin Anthony, Eugene Belilovsky, Irina Rish, and Timothée Lesort. Continual pre-training of large language models: How to (re)warm your model? *arXiv preprint arXiv:2308.04014*, 2023.
- Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A. Smith. Don’t stop pretraining: Adapt language models to domains and tasks. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8342–8360, 2020.
- Bobby He and Thomas Hofmann. Simplifying transformer blocks. In *International Conference on Learning Representations*, 2024.
- Adam Ibrahim, Benjamin Thérien, Kshitij Gupta, Mats L. Richter, Quentin Anthony, Timothée Lesort, Eugene Belilovsky, and Irina Rish. Simple and scalable strategies to continually pre-train large language models. *Transactions on Machine Learning Research*, 2024.
- Andrei Kanavalau, Carmen Amo Alonso, and Sanjay Lall. Gated removal of normalization in transformers enables stable training and efficient inference. *arXiv preprint arXiv:2602.10408v1*, 2026. Version 1. Later versions of this arXiv entry carry a different title; this paper follows version 1.

- Andrzej Karpathy. nanoGPT. <https://github.com/karpathy/nanoGPT>, 2022.
- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.
- Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, Dzmitry Bahdanau, Leandro von Werra, and Harm de Vries. The stack: 3 TB of permissively licensed source code. *arXiv preprint arXiv:2211.15533*, 2022.
- Daniël Lakens. Equivalence tests: A practical primer for t tests, correlations, and meta-analyses. *Social Psychological and Personality Science*, 8(4):355–362, 2017.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- Clare Lyle, Zeyu Zheng, Khimya Khetarpal, James Martens, Hado van Hasselt, Razvan Pascanu, and Will Dabney. Normalization and effective learning rates in reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- Michael McCloskey and Neal J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of Learning and Motivation*, volume 24, pages 109–165. Academic Press, 1989.
- Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. In *International Conference on Learning Representations*, 2018.
- Seyed Iman Mirzadeh, Mehrdad Farajtabar, Razvan Pascanu, and Hassan Ghasemzadeh. Understanding the role of training regimes in continual learning. In *Advances in Neural Information Processing Systems*, volume 33, 2020.
- Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018.
- Keiran Paster, Marco Dos Santos, Zhangir Azerbayev, and Jimmy Ba. OpenWebMath: An open dataset of high-quality mathematical web text. In *International Conference on Learning Representations*, 2024.
- Quang Pham, Chenghao Liu, and Steven C. H. Hoi. Continual normalization: Rethinking batch normalization for online continual learning. In *International Conference on Learning Representations*, 2022.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.
- Vinay Venkatesh Ramasesh, Aitor Lewkowycz, and Ethan Dyer. Effect of scale on catastrophic forgetting in neural networks. In *International Conference on Learning Representations*, 2022.
- Peter Todd. OpenTimestamps: Scalable, trust-minimized, distributed timestamping with Bitcoin. <https://petertodd.org/2016/opentimestamps-announcement>, 2016.
- Twan van Laarhoven. L2 regularization versus batch and weight normalization. *arXiv preprint arXiv:1706.05350*, 2017.
- Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture. In *Proceedings of the 37th International Conference on Machine Learning*, 2020.
- Linting Xue, Noah Constant, Adam Roberts, Mihir Kale, Rami Al-Rfou, Aditya Siddhant, Aditya Barua, and Colin Raffel. mT5: A massively multilingual pre-trained text-to-text transformer. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 483–498, 2021.

- Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- Guodong Zhang, Chaoqi Wang, Bowen Xu, and Roger Grosse. Three mechanisms of weight decay regularization. In *International Conference on Learning Representations*, 2019a.
- Hongyi Zhang, Yann N. Dauphin, and Tengyu Ma. Fixup initialization: Residual learning without normalization. In *International Conference on Learning Representations*, 2019b.
- Jiachen Zhu, Xinlei Chen, Kaiming He, Yann LeCun, and Zhuang Liu. Transformers without normalization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025.

A Evaluation schedule

Full development evaluations (2,097,152 labels per domain). Prefix: global updates $u = 763, 3052, 6104, 6409, 6714, 7019, 7324, 7629, 7934, 8239, 8544, 8849$ and 9156 . Continuation (each branch): $s \in \{0, 1, 10, 100\}$, every 305 updates from $s = 305$ to 6100 , and $s = 6104$.

Quick development evaluations (262,144 labels per domain). At the switch, and at continuation $s \in \{0, 1, 2, 5, 10, 20\}$, every 50 updates from $s = 50$ to 1000 , every 100 updates from $s = 1100$ to 6100 , and $s \in \{1525, 3050, 6104\}$ (80 points). Quick-dev values describe transients and feed matched adaptation. They never enter the full-dev primary contrast.

Diagnostics. Forward-only activation, gain and attention-entropy probes on the last 256 training windows of each domain, at the switch and at $s \in \{0, 10, 100, 1525, 6104\}$. The activation scales are analysed in Section 5.4 (exploratory); the other diagnostics are logged but not analysed here.

B Additional results

Table 14: Matched-adaptation differential forgetting (nats/token) and the interpolated Taper-minus continuation update s at which it reached each RMS code-CE target, with the observed bracket in parentheses.

Seed	Matched differential forgetting			Taper update at match (bracket)		
	$s=1525$	$s=3050$	$s=6104$	$s=1525$	$s=3050$	$s=6104$
101	-0.0075	-0.0038	-0.0308	1447 (1400–1500)	2740 (2700–2800)	5198 (5100–5200)
102	+0.0094	-0.0106	-0.0182	1475 (1400–1500)	2776 (2700–2800)	5775 (5700–5800)
103	-0.0259	+0.0145	-0.0286	1519 (1500–1525)	3024 (3000–3050)	5862 (5800–5900)

Table 15: Decomposition of the endpoint D (nats/token) into disjoint token classes (each class’s contribution weighted by its label share; the four contributions sum to D), and summaries of per-document effects D_i over the 4,409 web development documents. “Top-1% sum” is the signed contribution of the 1% of documents with the largest positive contributions.

Seed	Contribution to D by token class				Per-document D_i		
	W	A	P	C	Median	1%-trimmed	Top-1% sum
101	+0.0038	-0.0098	-0.0121	-0.0019	-0.0232	-0.0203	+0.0018
102	-0.0019	-0.0155	+0.0014	-0.0047	-0.0257	-0.0210	+0.0020
103	+0.0008	-0.0227	-0.0055	+0.0012	-0.0272	-0.0265	+0.0020
Labels	33,807	1,786,076	230,998	46,271	4,409 documents		

Table 16: Rare-token flag R (pre-specified; overlaps the four classes) and the exploratory document bootstrap of the endpoint D (10,000 replicates, documents resampled jointly across arms and seeds; conditional on the trained models).

Seed	R labels	Mean D on R	R contribution to D	Document-bootstrap 95% interval for D
101	5,267	-0.082	-0.0002	$[-0.0218, -0.0182]$
102	5,285	-0.206	-0.0005	$[-0.0226, -0.0189]$
103	5,147	-0.199	-0.0005	$[-0.0281, -0.0243]$
Mean of seeds				$[-0.0235, -0.0211]$

C Study 2: additional results

Table 17: Study 2, web $\rightarrow$ Chinese: decomposition of the endpoint D_X (nats/token) into disjoint token classes (the four contributions sum to D_X), summaries of per-document effects D_i over the 4,409 web development documents, and the contribution of the overlapping rare-token flag R.

Seed	Contribution to D_X by token class				Per-document D_i			R contribution
	W	A	P	C	Median	1%-trimmed	Top-1% sum	
101	-0.0046	-0.0088	-0.0730	-0.0022	-0.0639	-0.0885	+0.0020	+0.0007
102	-0.0013	+0.0462	-0.0417	+0.0033	+0.0102	+0.0066	+0.0036	-0.0006
103	-0.0049	-0.0261	-0.0131	+0.0089	-0.0351	-0.0351	+0.0021	-0.0003
104	+0.0025	-0.0233	-0.0288	-0.0017	-0.0510	-0.0512	+0.0019	+0.0001
105	+0.0024	+0.0020	-0.0187	-0.0036	-0.0148	-0.0182	+0.0028	-0.0003
106	-0.0016	-0.0115	+0.0069	-0.0027	-0.0142	-0.0093	+0.0043	+0.0004

Table 18: Study 2: share of updates with gradient clipping active (pre-clipping global norm > 1.0), mean over runs with the range in parentheses, by training phase. Prefix, web and Python rows cover the fresh seeds 104–106; the Chinese row covers all six seeds.

Phase	Runs	RMS	Taper-minus
Prefix, $u \leq 763$ (calibration)	3	15.9% (15.9–16.0)	15.9% (15.9–16.0)
Prefix, $764 \leq u \leq 6,103$	3	7.9% (7.1–9.5)	11.5% (8.9–13.4)
Prefix, $u \geq 6,104$	3	17.9% (13.4–24.3)	39.4% (21.2–56.3)
Web branch	3	89.8% (87.6–93.1)	92.0% (84.1–97.7)
Python branch	3	94.4% (92.7–95.7)	92.7% (90.5–95.3)
Chinese branch	6	99.5% (99.2–99.7)	99.7% (99.0–100.0)